

Sistema de Agendamento de Consultas Médicas

Relatório Técnico — Conceitos de Sistemas Operacionais

Disciplina: Sistemas Operacionais

Stack: Python 3.12 · FastAPI · SQLAlchemy · SQLite · reportlab

Plataforma: Linux x86_64 (compatível com Windows e macOS)

2026

Sumário

1. Visão Geral do Projeto
2. Arquitetura do Sistema
3. Conceito 1 — Processos e Threads
4. Conceito 2 — Sistema de Arquivos
5. Conceito 3 — Gerência de Memória (Cache)
6. Conceito 4 — Concorrência e Exclusão Mútua
7. Conceito 5 — Chamadas de Sistema
8. Conceito 6 — Operações de I/O (Relatórios)
9. Conceito 7 — Logging e Gerência de Dispositivos
10. Decisões Técnicas e Dificuldades
11. Endpoints da API
12. Referências

1. Visão Geral do Projeto

O Sistema de Agendamento de Consultas Médicas é uma API REST completa desenvolvida em Python com FastAPI, com foco na demonstração prática de conceitos fundamentais de Sistemas Operacionais. O sistema gerencia pacientes, médicos e consultas, aplicando diretamente conceitos como threads, locks, sistema de arquivos, chamadas de sistema, cache em memória e operações de I/O.

Linguagem	Python 3.12
Framework	FastAPI 0.115 + Uvicorn (ASGI)
Banco	SQLAlchemy 2.0 (async) + SQLite +aiosqlite
Concorrência	asyncio.Lock + threading.Thread + queue.Queue
Relatórios	reportlab (PDF) + csv stdlib (CSV)
Logging	logging + RotatingFileHandler (5 MB / 3 backups)

2. Arquitetura do Sistema

A arquitetura segue o padrão em camadas:

Camada	Módulo	Responsabilidade
Router	core/routers/	Recebe requisições HTTP, valida entrada
Service	core/services/	Regras de negócio + lock de agendamento
Repository	core/repositories/	Queries SQL via SQLAlchemy async
Model	core/models/	Entidades ORM (Patient, Doctor, Appointment)
Config	config/	Settings, Logger, Database, Platform
Concorrência	concorrecia/	Locks, Workers, Fila de tarefas
Storage	storage/	Cache em memória com TTL
Reports	reports/	Geração de PDF e CSV

Estrutura de diretórios:

```
sistema_agendamento/
+-- src/
|   +-- main.py           # App FastAPI + lifespan
|   +-- config/
|   |   +-- settings.py   # Paths por SO, encoding, diretórios
|   |   +-- logger.py     # RotatingFileHandler
|   |   +-- database.py   # Engine async + sessão
|   |   +-- platform_info.py # Chamadas de sistema (uname, getpid, os.access)
|   +-- core/
|   |   +-- models/       # Patient, Doctor, Appointment (SQLAlchemy)
|   |   +-- schemas/      # Pydantic – validação de entrada/saída
|   |   +-- repositories/ # Camada de acesso ao banco (queries)
|   |   +-- services/     # Regras de negócio + lock de agendamento
|   |   +-- routers/      # Endpoints HTTP
|   |   +-- errors.py     # Handler global de erros padronizados
|   +-- concorrancia/
|   |   +-- locks.py      # asyncio.Lock – exclusão mútua
|   |   +-- workers.py    # Thread daemon + queue.Queue
|   +-- storage/
|   |   +-- cache.py      # MemoryCache TTL + RLock + eviction thread
|   +-- reports/
|   |   +-- pdf_report.py  # Geração de PDF por médico
|   |   +-- csv_report.py  # Exportação geral em CSV
|   |   +-- report_service.py # Despacha geração para thread
+-- data/
|   +-- agendamento.db    # SQLite – banco principal
|   +-- logs/agendamento.log # Log rotacionado
|   +-- backups/          # Cópias do banco com timestamp
|   +-- relatorios/       # PDFs e CSVs gerados
|   +-- cache/            # Cache em disco (limpeza automática)
+-- run.py                # Ponto de entrada
+-- requirements.txt
```

3. Conceito 1 — Processos e Threads

O sistema opera em um único processo Python (um único PID atribuído pelo SO), mas usa múltiplas threads para separar responsabilidades e não bloquear o event loop do asyncio durante operações pesadas de I/O.

Threads em execução:

Thread	Tipo	Responsabilidade
MainThread	Principal	Event loop asyncio — atende requisições HTTP
BackgroundWorker	Daemon	Consome fila de tarefas (backup, limpeza)
PDFWorker-{id}	Daemon	Gera PDF de relatório por médico
CSVWorker-{id}	Daemon	Gera CSV de exportação geral
CacheCleanup-*	Daemon	Remove entradas expiradas do cache a cada 60s

Por que threads daemon?

Threads marcadas como `daemon=True` são encerradas automaticamente pelo SO quando o processo principal termina, sem necessidade de `join()` explícito. Isso garante que o servidor nunca fique travado aguardando workers ao encerrar.

Padrão Produtor/Consumidor:

```
# workers.py – fila thread-safe
_task_queue: queue.Queue = queue.Queue(maxsize=50)

# Produtor (event loop): enfileira sem bloquear
def enqueue(task_name: str) -> str:
    task_id = str(uuid.uuid4())[:8]
    _task_queue.put_nowait((task_id, task_name)) # não bloqueia
    return task_id

# Consumidor (BackgroundWorker thread): bloqueia em get() sem usar CPU
def _worker_loop():
    while True:
        task_id, task_name = _task_queue.get(timeout=1) # SO suspende a thread
        handler = _TASK_HANDLERS[task_name]
        result = handler()
        _task_queue.task_done()
```

4. Conceito 2 — Sistema de Arquivos

Toda a persistência não-relacional (logs, backups, relatórios, cache) usa o sistema de arquivos do SO diretamente, via pathlib e shutil.

Estrutura de diretórios criada na inicialização:

Em settings.py, o método `_ensure_directories()` cria os diretórios de dados com `pathlib.Path.mkdir(parents=True, exist_ok=True)` e aplica permissão 755 em sistemas Unix via `os.chmod(d, 0o755)`. No Windows, `os.chmod` é ignorado silenciosamente pelo Python.

Backup com metadados preservados:

```
# workers.py - _task_backup()
# shutil.copy2 preserva atime e mtime do inode original
shutil.copy2(db_path, dest)

# Mantém apenas os 5 backups mais recentes
backups = sorted(settings.BACKUPS_DIR.glob("agendamento_*.db"))
for old in backups[:-5]:
    old.unlink() # libera espaço no filesystem
```

Paths específicos por SO:

SO	Convenção de dados	Encoding
Linux	~/.local/share/sistema_agendamento/ (XDG Base Dir Spec)	utf-8
macOS	~/Library/Application Support/ SistemaAgendamento/	utf-8
Windows	%APPDATA%\SistemaAgendamento\	utf-8-sig (BOM p/ Excel)

5. Conceito 3 — Gerência de Memória (Cache)

O módulo `storage/cache.py` implementa um cache em memória com TTL (Time-To-Live) e eviction automática, análogo ao page aging em algoritmos de substituição de páginas do SO.

```
# cache.py – estrutura de entrada
class _CacheEntry:
    __slots__ = ("value", "expires_at")

    def __init__(self, value, ttl):
        self.value = value
        # time.monotonic(): imune a mudanças de relógio do SO (NTP, DST)
        self.expires_at = time.monotonic() + ttl

    def is_expired(self):
        return time.monotonic() > self.expires_at

# Eviction automática em thread daemon (a cada 60s):
def _evict_expired(self):
    with self._lock:          # RLock: reentrante, thread-safe
        expired = [k for k, e in self._store.items() if e.is_expired()]
        for k in expired:
            del self._store[k]  # libera referência → GC do Python coleta
```

Três instâncias independentes (`appointment_cache`, `doctor_cache`, `patient_cache`) permitem invalidação seletiva por domínio sem afetar os demais.

6. Conceito 4 — Concorrência e Exclusão Mútua

O problema central de concorrência no sistema é a race condition no agendamento: duas requisições simultâneas poderiam passar na verificação de conflito de horário ao mesmo tempo e criar duas consultas sobrepostas.

Solução: `asyncio.Lock`

Um `asyncio.Lock` envolve a seção crítica (verificação + inserção), garantindo que apenas uma corrotina por vez execute o bloco. Ao contrário de `threading.Lock` — que bloquearia toda a thread do event loop durante a espera — `asyncio.Lock` cede o controle ao event loop, permitindo que outras requisições continuem sendo processadas.

```
# appointment_service.py – seção crítica
async with appointment_lock():          # asyncio.Lock
    # Nenhuma outra corrotina entra aqui enquanto o lock está adquirido
    if await self.appointments.doctor_has_conflict(...):
        raise HTTPException(409, "Médico já possui consulta neste horário.")
    if await self.appointments.patient_has_conflict(...):
        raise HTTPException(409, "Paciente já possui consulta neste horário.")
    appointment = await self.appointments.create(data)
# Lock liberado – próxima corrotina na fila pode prosseguir
```

Limitação documentada: `asyncio.Lock` opera dentro de um único processo. Em deploy com múltiplos workers (Gunicorn multiprocess), seria necessário um lock distribuído (ex: Redis SETNX / Redlock) para coordenar entre processos.

7. Conceito 5 — Chamadas de Sistema

O módulo `config/platform_info.py` coleta informações do SO via syscalls e APIs de plataforma, expondo os dados no endpoint GET `/system/info`.

Syscall / API	Python	Informação obtida
<code>uname(2)</code>	<code>platform.uname()</code>	Nome, release, versão, máquina do SO
<code>getpid(2)</code>	<code>os.getpid()</code>	PID do processo atual
<code>getppid(2)</code>	<code>os.getppid()</code>	PID do processo pai (shell/supervisor)
<code>sched_getaffinity(2)</code>	<code>os.cpu_count()</code>	Núcleos lógicos disponíveis
<code>access(2)</code>	<code>os.access(path, os.W_OK)</code>	Permissões reais do processo
<code>stat(2)</code>	<code>Path.stat().st_size</code>	Tamanho e timestamps do inode
<code>clock_gettime(2)</code>	<code>time.monotonic()</code>	Relógio monotônico do kernel
<code>environ</code>	<code>os.environ.get('XDG_...')</code>	Variáveis de ambiente do processo

8. Conceito 6 — Operações de I/O (Relatórios)

A geração de relatórios é a operação de I/O mais intensa do sistema. Por isso é delegada a threads de background, retornando 202 Accepted imediatamente ao cliente enquanto o arquivo é produzido.

Fluxo de geração de PDF:



Encoding específico por plataforma no CSV:

```
# csv_report.py
with open(filepath, "w",
          newline="",
          encoding=settings.ENCODING # evita \r\r\n no Windows
          ) as f:
    writer = csv.writer(f)
    ...
```

9. Conceito 7 — Logging e Gerência de Dispositivos

Todas as operações do sistema são registradas em arquivo de log com rotação automática, usando o módulo logging da stdlib com RotatingFileHandler.

```
# logger.py
file_handler = RotatingFileHandler(
    filename = settings.LOGS_DIR / "agendamento.log",
    maxBytes = 5 * 1024 * 1024, # rotaciona ao atingir 5 MB
    backupCount = 3,           # mantém agendamento.log.1, .2, .3
    encoding = settings.ENCODING,
)
# Formato: timestamp ISO (hora local do SO) | nível | módulo | mensagem
formatter = logging.Formatter(
    fmt = "%(asctime)s | %(levelname)-8s | %(name)s | %(message)s",
    datefmt = "%Y-%m-%d %H:%M:%S", # hora local – syscall gettimeofday(2)
)
```

A rotação de logs é um conceito de gerência de dispositivos: quando o arquivo atinge 5 MB, o SO cria um novo inode para agendamento.log e renomeia o anterior para agendamento.log.1, liberando espaço de forma controlada.

10. Decisões Técnicas e Dificuldades

asyncio.Lock vs threading.Lock

Durante o desenvolvimento, `threading.Lock` foi testado primeiro na seção crítica de agendamento. O teste de race condition revelou que ele bloqueava toda a thread do event loop durante a espera, impedindo outras requisições. A substituição por `asyncio.Lock` resolveu o problema: a corrotina cede o controle ao event loop enquanto aguarda o lock.

aiosqlite para I/O assíncrono

SQLAlchemy com driver síncrono bloquearia a thread do event loop a cada query. `aiosqlite` delega a operação ao SO e usa `await` para liberar o loop, permitindo que outras requisições sejam atendidas durante a espera por I/O de disco.

Pacote 'concurrent' conflitando com stdlib

O diretório `src/concurrent/` conflitava com o módulo `stdlib concurrent.futures`, causando `ImportError`. A solução foi renomear para `src/concorrenca/` — nome mais semântico em português e sem conflito com a biblioteca padrão.

Threads daemon para workers

Workers marcados como `daemon=True` são encerrados automaticamente com o processo principal, sem necessidade de `join()` explícito. Isso evita que o servidor fique travado aguardando tarefas em background ao receber `SIGTERM`.

time.monotonic() no cache

`time.time()` é sensível a mudanças de relógio do sistema (NTP, horário de verão). `time.monotonic()` usa o relógio monotônico do kernel, garantindo que o TTL do cache nunca expire prematuramente por ajuste de relógio.

11. Endpoints da API

Pacientes

Método	Rota	Descrição
POST	/patients/	Cria paciente (valida CPF único)
GET	/patients/	Lista pacientes (paginado)
GET	/patients/{id}	Busca por ID
GET	/patients/cpf/{cpf}	Busca por CPF
PUT	/patients/{id}	Atualiza paciente
DELETE	/patients/{id}	Remove paciente

Médicos

Método	Rota	Descrição
POST	/doctors/	Cria médico (valida CRM único)
GET	/doctors/	Lista médicos
GET	/doctors/{id}	Busca por ID
GET	/doctors/crm/{crm}	Busca por CRM
GET	/doctors/specialty/{s}	Filtra por especialidade
PUT	/doctors/{id}	Atualiza médico
DELETE	/doctors/{id}	Remove médico

Consultas

Método	Rota	Descrição
POST	/appointments/	Cria consulta (lock + regras de negócio)
GET	/appointments/	Lista consultas
GET	/appointments/{id}	Busca por ID
GET	/appointments/doctor/{id}	Consultas por médico
GET	/appointments/patient/{id}	Consultas por paciente
PUT	/appointments/{id}	Atualiza / reagenda
DELETE	/appointments/{id}	Remove consulta

Sistema (SO)

Método	Rota	Descrição
GET	/system/info	Informações do SO (uname, PID, CPUs, paths)
GET	/system/threads	Estado das threads e lock de agendamento
GET	/system/cache	Estatísticas do cache em memória
GET	/system/permissions	Permissões nos diretórios de dados
POST	/system/backup	Dispara backup do banco em background

POST	/system/cleanup	Limpeza de cache expirado em background
GET	/system/task/{id}	Resultado de tarefa assíncrona

Relatórios

Método	Rota	Descrição
POST	/reports/pdf/doctor/{id}	Gera PDF por médico (thread background)
POST	/reports/csv	Gera CSV geral (thread background)
GET	/reports/task/{id}	Consulta resultado + download_url
GET	/reports/download/{file}	Download do arquivo (FileResponse)
GET	/reports/list	Lista relatórios disponíveis

12. Referências

- [1] Python Software Foundation. `threading` — Thread-based parallelism. docs.python.org
- [2] Python Software Foundation. `asyncio` — Asynchronous I/O. docs.python.org
- [3] Python Software Foundation. `queue` — A synchronized queue class. docs.python.org
- [4] Python Software Foundation. `logging.handlers` — `RotatingFileHandler`. docs.python.org
- [5] Python Software Foundation. `os` — Miscellaneous operating system interfaces. docs.python.org
- [6] SQLAlchemy Documentation. Asyncio Platform. docs.sqlalchemy.org
- [7] FastAPI Documentation. Concurrency and `async/await`. fastapi.tiangolo.com
- [8] Tanenbaum, A. S. *Sistemas Operacionais Modernos*. 4. ed. Pearson, 2016.
- [9] Silberschatz, A.; Galvin, P. B.; Gagne, G. *Operating System Concepts*. 10. ed. Wiley, 2018.
- [10] XDG Base Directory Specification. freedesktop.org/wiki/Specifications/xdg-base-dir-spec
- [11] Reportlab User Guide. reportlab.com/docs/reportlab-userguide.pdf